

Universidad de Oriente — UNIVO

Desarrollo de aplicaciones móviles

Consumo del Servicio de Login

Recurrly — Expo / React Native

Clase: 7 — 10/08/2026

Actividad: Consumo del servicio de login

Patrón: MVVM

Cliente HTTP: Axios

API: <https://api-ecommerce-5aby.onrender.com>

Endpoint: POST /auth/login

Rama: feat/consumo-servicio-login

Androso

6 de septiembre de 2026

1 Objetivo

Implementar el inicio de sesión contra el API REST remoto usando Axios, organizar la solución con MVVM y comprobar en consola que el token de sesión llega correctamente. También se documenta el caso de error del servidor.

La clase (Notion: *Clase 7 — Consumo del servicio de login*) pide exactamente esta separación: la vista no carga la lógica; `useLogin` aísla estado, validación y llamada HTTP; `auth.service.ts` centraliza el consumo de `POST /auth/login`.

2 Arquitectura MVVM

Capa	Archivo	Responsabilidad
Model	<code>src/auth/models/auth.model.ts</code>	Interfaces <code>Role</code> , <code>UserResponse</code> y <code>AuthResponse</code> alineadas a la respuesta real del backend (<code>token + userResponse</code>).
Service	<code>src/auth/service/auth.service.ts</code>	<code>AuthService.login</code> con <code>axios.post</code> y mapeo de <code>error.response.data.message</code> .
ViewModel	<code>src/auth/viewmodels/use-login.ts</code>	Estado, validación, <code>isLoading</code> , impresión del token y manejo de error.
View	<code>app/(public)/index.tsx</code>	Pantalla de login: campos, <code>AuthError</code> y <code>ActivityIndicator</code> .

Los nombres del JSON real del API son `userResponse` y `lastName` (no `user/lastname` de las notas de clase). El modelo TypeScript sigue el contrato verificado en Postman/curl.

3 Consumo del endpoint

Base URL: `EXPO_PUBLIC_API_URL` o, por defecto, `https://api-ecommerce-5aby.onrender.com`.

Cuerpo enviado:

```
{ "email": string, "password": string }
```

Respuesta exitosa: HTTP 200 con JWT en `token` (prefijo Bearer) y datos de usuario en `userResponse`.

Error de credenciales: HTTP 401

```
{"statusCode":401,"message":"El email o Password no son validos"}
```

En el ViewModel, tras un login correcto se ejecuta:

```
console.log('[Auth] Session token received:', session.token);
```

y en fallo:

```
console.error('[Auth] Login failed:', message);
```

4 Caso de error del servidor

Credenciales inválidas. El API responde 401 y la vista muestra el mensaje del servidor.

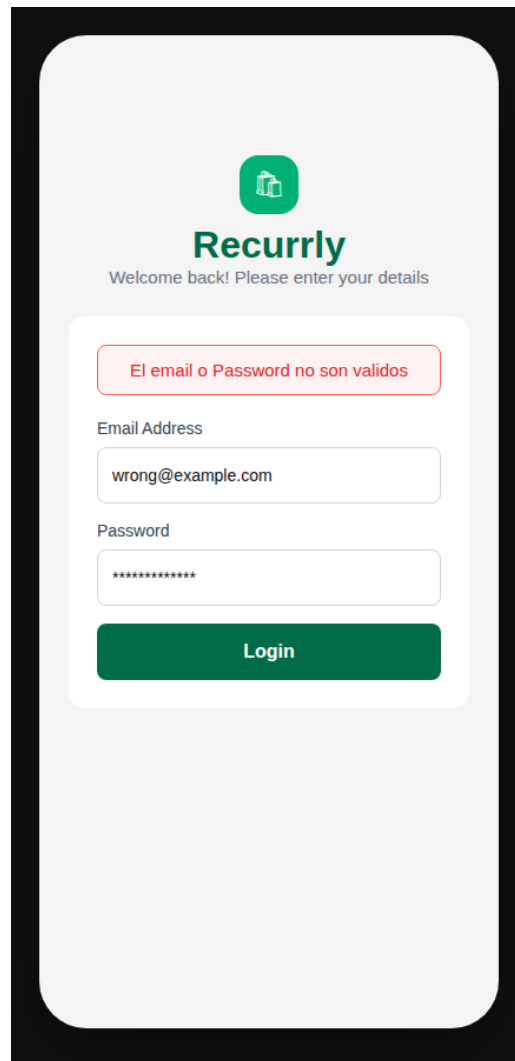


Figure 1: Pantalla de login mostrando el error del API.

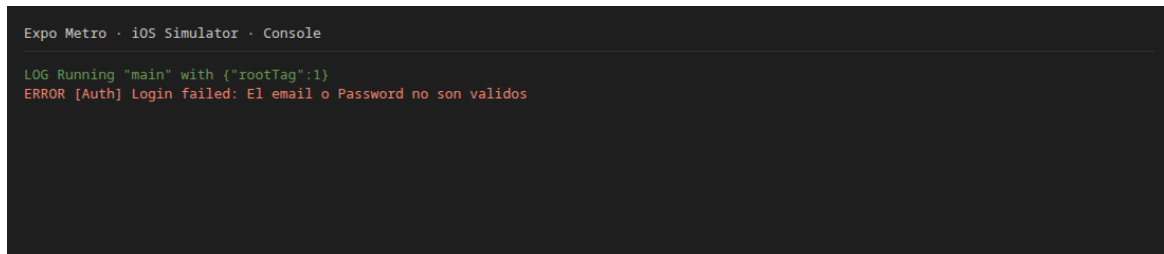
5 Caso de éxito: token de sesión

Login correcto con usuario de prueba. El token se imprime en la consola de Metro, tal como pide la clase (`handleLogin` imprime el token después de `AuthService.login()`).

Usuario de verificación: `recurrly.login.1788667165@example.com` (id 352). El JWT llega con el prefijo `Bearer`.

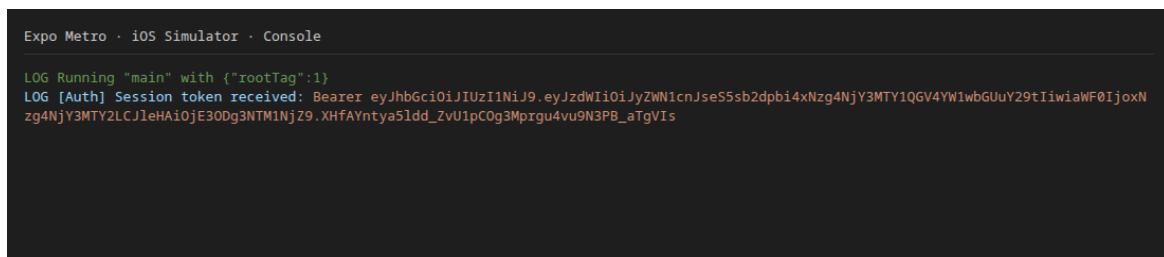
6 Conclusión

El servicio de login queda consumido con Axios, tipado, separado en MVVM y con evidencia de error 401 y de token en consola. La vista solo presenta; el hook orquesta; el servicio habla con Render.



A screenshot of the Expo Metro console output. The title bar reads 'Expo Metro · iOS Simulator · Console'. The log shows a green line: 'LOG Running "main" with {"rootTag":1}'. This is followed by a red line: 'ERROR [Auth] Login failed: El email o Password no son validos'.

Figure 2: Salida de depuración del error de login.



A screenshot of the Expo Metro console output. The title bar reads 'Expo Metro · iOS Simulator · Console'. The log shows a green line: 'LOG Running "main" with {"rootTag":1}'. This is followed by another green line: 'LOG [Auth] Session token received: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJyZW51cnJseS5sb2dpbi4xNzg4NjY3MTY1QGV4YW1wbGUuY29tIiwiaWF0IjoxNzg4NjY3MTY2LCJleHAiOjE3ODg3NTM1NjZ9.XHfAYntya5ldd_ZvU1pCOg3Mprgu4vu9N3PB_aTgVIs'.

Figure 3: Token de sesión recibido e impreso en consola.